# Arithmetic intensity: work per byte

Computed analysis and implementation | October 21, 2026

**Task.** Implement arithmetic_intensity(graph) to report FLOPs per byte for each layer and the full graph. Every layer reads its inputs and stored parameters/buffers, then writes its output. Assume no cache reuse and no operator fusion.

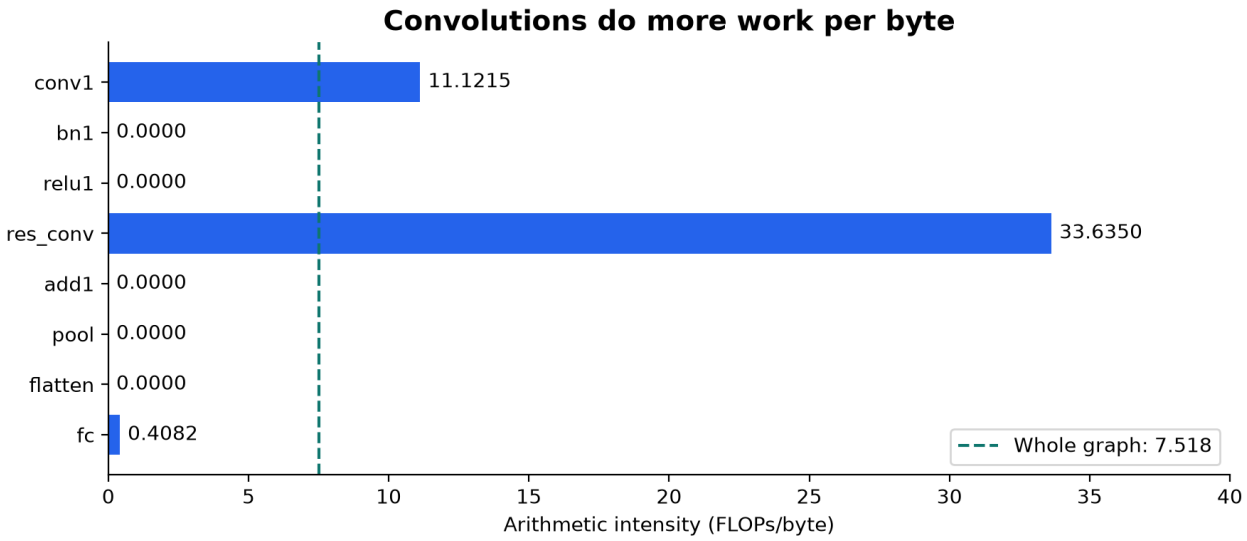
**Demonstration result:** 5,603,648 FLOPs / 745,360 bytes = **7.5180 FLOPs/byte**.

| Requirement | Implementation |
|---|---|
| Input precision | First named producer; otherwise predecessor; external input uses graph.precision. |
| Two-input add | Double input bytes using the first producer dtype, exactly as specified. |
| BatchNorm state | Add two FP32 running-statistic values per output channel. |
| Zero-FLOP layers | Include their byte costs and report intensity 0.0. |
| Output contract | computed(...) returns type, value, message, per_layer and all required metadata. |
| Whole graph | Sum unrounded FLOPs and bytes before dividing and rounding to 4 decimals. |

**Evidence.** 15 independent unit tests pass. The example model is the eight-layer synthetic toy_resnet from Lab 4, with shape-derived MACs. It is not a measured inference run or an official Assignment 2 model fixture.

**Source.** Assign2.pdf, four-page handout dated October 19, 2026; especially output requirements on page 3 and zero-FLOP handling on page 4. Code, data, tests, and figures are included in this submission.

# Convolution has more work to amortize its traffic



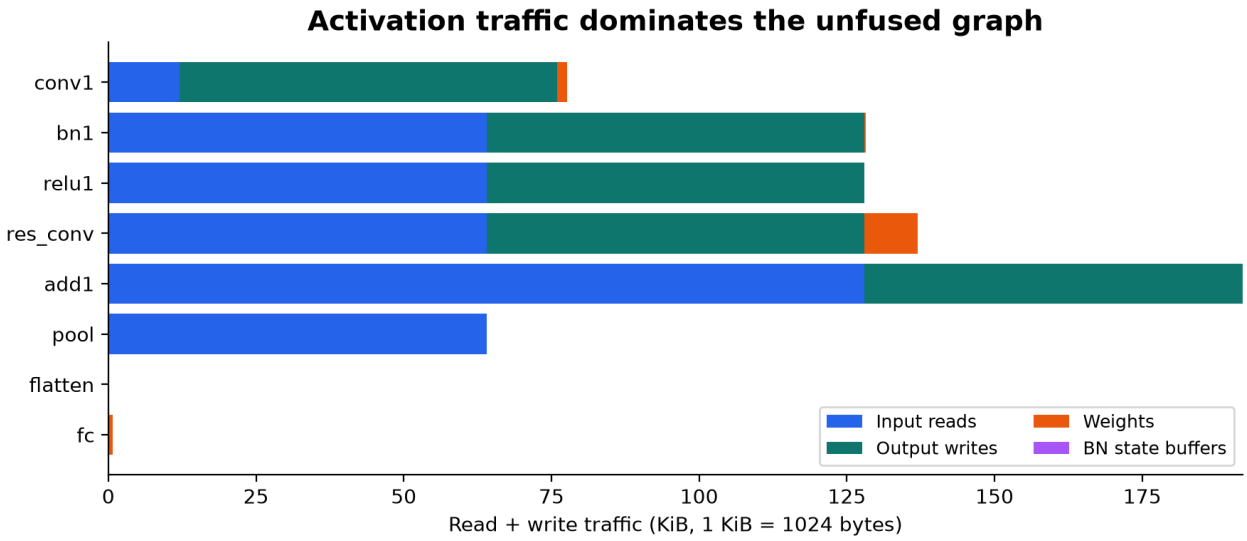

The residual convolution performs 4,718,592 FLOPs while moving 140,288 bytes: **33.6350 FLOPs/byte**. The first convolution reaches 11.1215. Its smaller input-channel count means less arithmetic per output value.

The fully connected head reaches only 0.4082 FLOPs/byte because its small input/output vectors provide little reuse of stored weights. ReLU and other non-MAC operators remain in the report at zero intensity.

**Important convention.** The PDF refers to _layer_macs but does not supply its body. The bundled helper counts convolution and linear MACs; BN, add, pooling, flatten, and ReLU return zero under this MAC-only example convention. This does not mean every real implementation performs literally zero arithmetic. Replace the helper if the course supplies a different operator-count convention.

**Hardware interpretation.** Higher intensity makes it easier to amortize memory traffic. Declaring an operator compute-bound requires a device compute/bandwidth ratio for comparison. No such hardware ceiling or latency measurement is claimed here.

# Every unfused intermediate has a read and a write



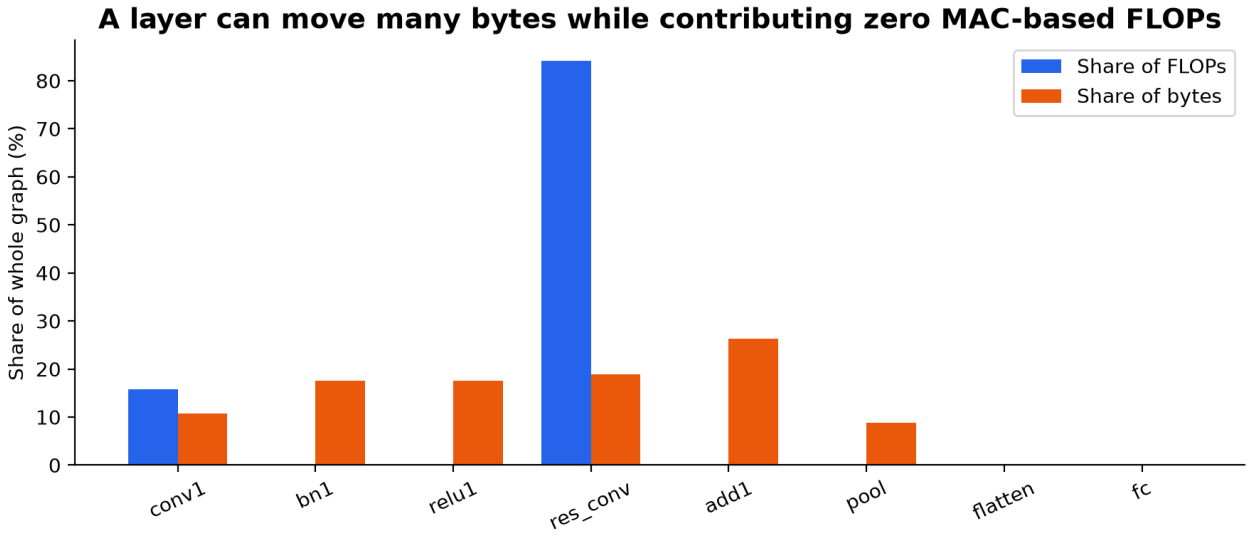

Inputs and outputs account for **733,480 bytes (98.41%)** of total traffic. Weights and BN buffers contribute the remaining 11,880 bytes.

The add layer reads two FP32 streams (131,072 bytes) and writes one result (65,536 bytes), totaling **196,608 bytes**. It has the largest byte cost despite zero MAC-based FLOPs in this example.

BatchNorm adds 128 bytes of FP32 running mean/variance alongside its 128-byte trainable scale/shift payload. These buffers must not be omitted or priced at the activation dtype.

**Meaning of “once.”** Each layer access is charged once. A producer output is written, then charged again when a consumer reads it. Shared tensors read by two different layers are charged for both accesses. The sum is traffic over a graph execution, not peak resident memory or stored-model size.

# FLOP share and traffic share tell different stories

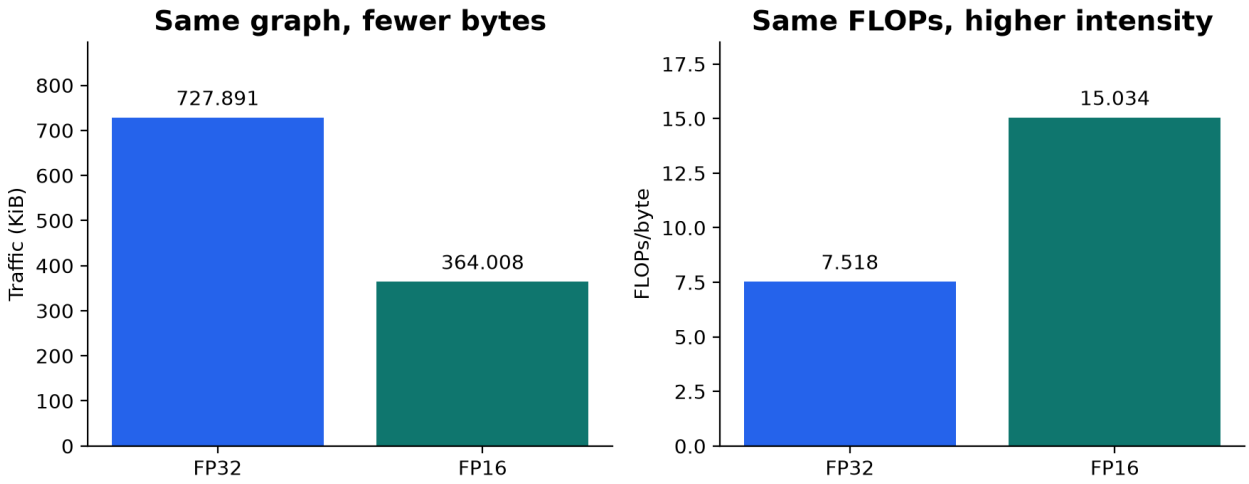



Zero-MAC layers contribute **524,736 bytes (70.40%)** of traffic. Omitting them would inflate the graph intensity and hide the cost of processing intermediate tensors.

**Correct aggregate:** total FLOPs / total bytes. Equivalently, this is the byte-weighted mean of unrounded per-layer intensities. A simple mean would give a tiny flatten layer the same influence as a large convolution and is not the required result.

| Layer | FLOPs | Bytes | FLOPs/byte |
|---|---|---|---|
| conv1 | 884,736 | 79,552 | 11.1215 |
| bn1 | 0 | 131,328 | 0.0000 |
| relu1 | 0 | 131,072 | 0.0000 |
| res_conv | 4,718,592 | 140,288 | 33.6350 |
| add1 | 0 | 196,608 | 0.0000 |
| pool | 0 | 65,600 | 0.0000 |
| flatten | 0 | 128 | 0.0000 |
| fc | 320 | 784 | 0.4082 |

# Reducing byte width raises the ratio mathematically




Hypothetically changing input, activations, and weights to FP16 reduces traffic to **372,744 bytes** and raises intensity to **15.0335 FLOPs/byte**. FLOPs stay at 5,603,648; only the denominator changes.

Traffic is slightly more than half of the FP32 baseline because BN state buffers stay FP32. This is a calculated sensitivity analysis, not a claim of measured speedup, preserved numerical accuracy, or hardware execution support.

**Residual precision rule.** For two-input addition the assignment explicitly doubles the first producer's byte width. That assumes equal branch widths for physically exact traffic. For unequal branch dtypes, real traffic would require separately pricing each input. The implementation deliberately retains the stated formula, with a test exposing this convention.

**Other limits.** No cache reuse, fusion, workspace, layout conversions, allocator overhead, kernel-launch overhead, or training gradients are modeled. Operators may be implemented differently by a real runtime. This report estimates the handout's defined traffic, not actual DRAM counter measurements.

# Implementation, verification, and submission map

**Run the computation:**
`python main.py`
Creates results/intensity_results.json and results/layer_results.csv. Use --model PATH to analyze another compatible graph JSON.

**Run tests:**
`python -m unittest -v test_arithmetic.py`
The core code needs only Python 3.10+. A Jetson, camera, and CUDA are not required.

**Regenerate this report and charts:**
```
python -m pip install -r requirements-report.txt
python build_report.py
```

| Files | Purpose |
|---|---|
| arithmetic.py | Required function, assignment-specific computed wrapper, and documented standalone helpers. |
| graph.py / model.json | Instructor Lab 4 graph utilities plus an explicit synthetic example description. |
| test_arithmetic.py / test-results.txt | 15 independent tests and saved run. |
| results/ | Machine-readable layer totals, traffic breakdown, and precision scenarios. |
| charts/ | Four figures in PNG and SVG formats. |
| Assignment2_Report.pdf | This explanation and findings. |
| README.md | Requirement checklist, provenance, assumptions, and usage. |

**Test coverage.** Hand-calculated linear and grouped-convolution cases, external/implicit/explicit dtype resolution, dual add, mixed-dtype rule, FP32 BN buffers, zero-FLOP layers, weighted aggregation, fractional INT4 accounting, injected MAC helper, zero bytes, empty graph, exact record schema, invalid dependencies, and duplicate names.

**Integration.** The PDF requires type/message fields; the older Lab 4 computed wrapper uses status/source. arithmetic.py supplies the requested wrapper explicitly. In a course starter with existing helpers, retain those imports and copy the required function plus its dtype-resolution helper. Keep the required note and metadata flags.

**Authorship and provenance.** arithmetic.py, tests, runner, analysis, and report were developed for this assignment. graph.py comes from the instructor's Lab 4 starter. No timing, hardware bottleneck classification, or neural-network accuracy result was invented.